

Performance Testing

Date	04 JULY 2026
Team ID	
Project Name	A Comprehensive Measure of Well-Being
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Apache JMeter
Type of Testing	Load Testing and Stress Testing
Target Module / API	HDI Prediction Form and Prediction Endpoint
Test Environment	Local Flask Development Server
Test Date	04 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Load Home Page	10	60	Page loads successfully without errors
2	Access Predict HDI Page	25	60	Prediction form loads within 2 seconds
3	Submit HDI Prediction Request	50	120	HDI score and category are generated successfully
4	Peak Load Prediction Testing	100	120	Application remains stable under high load

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.82 seconds	Pass	Fast average response
2	Response Time (Max)	< 5 seconds	2.41 seconds	Pass	Within acceptable limit
3	Throughput (Req/sec)	> 5 Req/sec	> 5 Req/sec	Pass	Good request handling
4	Error Rate	< 1%	0.0%	Pass	No request errors
5	CPU Utilization	< 80%	42%	Pass	Within safe limit
6	Memory Utilization	< 80%	58%	Pass	Stable memory usage

Step 4: Observations & Analysis

Key Findings:

The HDI Predictor was tested under different user loads to evaluate response time, throughput, stability, and error rate. The application successfully processed prediction requests and returned the predicted HDI score and category. The final performance assessment was based on the actual results recorded during testing.

Bottlenecks Identified:

Any increase in response time under higher concurrent user loads should be recorded here based on the actual test results. Local development server limitations may affect performance during peak-load testing.

Optimization Steps Taken:

The trained machine learning model is loaded efficiently for prediction, input processing is kept lightweight, and application components are organized to reduce unnecessary processing. Further performance improvements can be achieved by using a production web server and cloud deployment.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

